```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

```
!nvidia-smi
```

```
Fri Sep 25 06:09:26 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   43C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
!pip install -U transformers datasets peft trl bitsandbytes accelerate
```

Show hidden output

```
import torch
from datasets import load_dataset
from transformers import (
AutoTokenizer,
AutoModelForCausalLM,
BitsAndBytesConfig
)
from peft import LoraConfig
```

```
print(torch.cuda.is_available())
```

```
True
```

```
!pip install -U datasets
```

Show hidden output

```
from datasets import load_dataset
dataset = load_dataset("stanfordnlp/imdb")
```

```
README.md: 100%  7.81k/7.81k [00:00<00:00, 109kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
plain_text/train-00000-of-00001.parquet: reconstructing file: 100%  21.0MB / 21.0MB, 2.00MB/s
plain_text/train-00000-of-00001.parquet: downloading bytes:  20.9MB, 1.97MB/s
plain_text/test-00000-of-00001.parquet: reconstructing file: 100%  20.5MB / 20.5MB, 1.97MB/s
plain_text/test-00000-of-00001.parquet: downloading bytes:  20.4MB, 1.94MB/s
plain_text/unsupervised-00000-of-00001.p(…): reconstructing file: 100%  42.0MB / 42.0MB, 4.04MB/s
plain_text/unsupervised-00000-of-00001.p(…): downloading bytes:  41.9MB, 3.98MB/s
Generating train split: 100%  25000/25000 [00:00<00:00, 87777.41 examples/s]
Generating test split: 100%  25000/25000 [00:00<00:00, 90636.08 examples/s]
Generating unsupervised split: 100%  50000/50000 [00:00<00:00, 65089.85 examples/s]
```

```
print(dataset)
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
```

```
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
```

```
print(dataset["train"][0])
```

```
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was fir
```

```
print(dataset["train"].features)
```

```
{'text': Value('string'), 'label': ClassLabel(names=['neg', 'pos'])}
```

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(2000))
test_dataset = dataset["test"].shuffle(seed=42).select(range(500))
```

```
print("Training:", len(train_dataset))
print("Testing:", len(test_dataset))
```

```
Training: 2000
Testing: 500
```

```
def format_example(example):
  sentiment = "Positive" if example["label"] == 1 else "Negative"
  return {
    "text": f"""Review:
{example["text"]}
Sentiment:
{sentiment}"""
}
```

```
train_dataset = train_dataset.map(format_example)
test_dataset = test_dataset.map(format_example)
```

```
Map: 100%  2000/2000 [00:00<00:00, 6556.14 examples/s]
Map: 100%  500/500 [00:00<00:00, 7266.87 examples/s]
```

```
print(train_dataset[0]["text"])
```

```
Review:
There is no relation at all between Fortier and Profiler but the fact that both are police series about violent crimes. Prof
Sentiment:
Positive
```

```
!pip install -U transformers peft trl bitsandbytes accelerate
```

Show hidden output

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
```

```
print(torch.cuda.is_available())
```

```
True
```

```
bnb_config = BitsAndBytesConfig(
load_in_4bit=True,
bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.float16,
bnb_4bit_use_double_quant=True
)
```

```
model_name = "Qwen/Qwen2.5-1.5B-Instruct"
model = AutoModelForCausalLM.from_pretrained(
model_name,
quantization_config=bnb_config,
device_map="auto"
)
```

```
config.json: 100%  660/660 [00:00<00:00, 63.8kB/s]
model.safetensors: reconstructing file: 100%  3.09GB / 3.09GB, 120MB/s
model.safetensors: downloading bytes:  2.69GB, 156MB/s
Loading weights: 100%  338/338 [00:05<00:00, 153.80it/s]
generation_config.json: 100%  242/242 [00:00<00:00, 5.63kB/s]
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
```

```
tokenizer_config.json: 100%  7.30k/7.30k [00:00<00:00, 307kB/s]
vocab.json: 100%  2.78M/2.78M [00:00<00:00, 24.3MB/s]
merges.txt: 100%  1.67M/1.67M [00:00<00:00, 22.5MB/s]
tokenizer.json: 100%  7.03M/7.03M [00:00<00:00, 40.7MB/s]
```

```
print("Tokenizer loaded successfully!")
print("Pad token:", tokenizer.pad_token)
```

```
Tokenizer loaded successfully!
Pad token: <|im_end|>
```

```
from peft import prepare_model_for_kbit_training
model = prepare_model_for_kbit_training(model)
print("Model prepared for QLoRA training!")
```

```
Model prepared for QLoRA training!
```

```
from peft import LoraConfig
lora_config = LoraConfig(
r=16,
lora_alpha=32,
lora_dropout=0.05,
bias="none",
task_type="CAUSAL_LM",
target_modules=[
"q_proj",
"k_proj",
"v_proj",
"o_proj"
]
)
print("LoRA configuration created!")
```

```
LoRA configuration created!
```

```
import trl
print("TRL version:", trl.__version__)
```

```
TRL version: 1.14.0
```

```
from trl import SFTConfig
training_args = SFTConfig(
output_dir="./qwen_imdb_qlora",
num_train_epochs=1,
per_device_train_batch_size=2,
gradient_accumulation_steps=4,
learning_rate=2e-4,
logging_steps=10,
save_steps=100,
fp16=True,
report_to="none",
max_length=512
)
print("Training configuration created!")
```

```
Training configuration created!
```

```
from trl import SFTTrainer
trainer = SFTTrainer(
model=model,
args=training_args,
train_dataset=train_dataset,
processing_class=tokenizer,
```

```
peft_config=lora_config,
)
print("SFT Trainer created successfully!")
```

```
Adding EOS to train dataset: 100%  2000/2000 [00:00<00:00, 11422.87 examples/s]
Tokenizing train dataset: 100%  2000/2000 [00:05<00:00, 464.99 examples/s]
Building labels for train dataset: 100%  2000/2000 [00:02<00:00, 1295.18 examples/s]
Truncating train dataset: 100%  2000/2000 [00:01<00:00, 1322.92 examples/s]
Dropping fully masked examples from train dataset: 100%  2000/2000 [00:00<00:00, 2063.28 examples/s]
SFT Trainer created successfully!
```

```
import torch
print("BF16 supported:", torch.cuda.is_bf16_supported())
print("FP16 available:", torch.cuda.is_available())
```

```
BF16 supported: True
FP16 available: True
```

```
from trl import SFTConfig
training_args = SFTConfig(
output_dir="./qwen_imdb_qlora",
num_train_epochs=1,
per_device_train_batch_size=2,
gradient_accumulation_steps=4,
learning_rate=2e-4,
logging_steps=10,
save_steps=100,
bf16=True,
fp16=False,
report_to="none",
max_length=512
)
print("Fixed training configuration created!")
```

```
Fixed training configuration created!
```

```
from trl import SFTTrainer
trainer = SFTTrainer(
model=model,
args=training_args,
train_dataset=train_dataset,
processing_class=tokenizer,
peft_config=lora_config,
)
print("Trainer recreated successfully!")
```

```
/usr/local/lib/python3.13/dist-packages/peft/mapping_func.py:148: UserWarning: You are trying to modify a model with PEFT fo
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/peft/tuners/tuners_utils.py:331: UserWarning: Already found a `peft_config` attribut
  warnings.warn(
Trainer recreated successfully!
```

```
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained(
model_name,
quantization_config=bnb_config,
device_map="auto"
)
model = prepare_model_for_kbit_training(model)
print("Clean Qwen model loaded and prepared!")
```

```
Loading weights: 100%  338/338 [00:06<00:00, 34.36it/s]
Clean Qwen model loaded and prepared!
```

```
trainer = SFTTrainer(
model=model,
args=training_args,
train_dataset=train_dataset,
processing_class=tokenizer,
peft_config=lora_config,
)
print("Clean trainer created successfully!")
```

```
Clean trainer created successfully!
```

```
trainer.train()
```

```
[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model
```

[59/250 12:45 < 42:45, 0.07 it/s, Epoch 0.23/1]

Step	Training Loss
10	3.208742
20	3.078950
30	3.001978
40	3.007062
50	2.960383

[250/250 54:36, Epoch 1/1]

Step	Training Loss
10	3.208742
20	3.078950
30	3.001978
40	3.007062
50	2.960383
60	2.938168
70	3.009289
80	2.955484
90	3.015396
100	3.003967
110	2.968313
120	2.971460
130	2.994522
140	2.932148
150	2.977146
160	3.009302
170	2.980818
180	2.989265
190	2.969207
200	2.979655
210	2.942854
220	2.956693
230	2.965497
240	2.982401
250	2.911676

```
TrainOutput(global_step=250, training_loss=2.9884149017333983, metrics={'train_runtime': 3288.7958,
'train_samples_per_second': 0.608, 'train_steps_per_second': 0.076, 'total_flos': 5377649033975808.0, 'train_loss':
2.9884149017333983, 'epoch': 1.0})
```

```
adapter_path = "./qwen_imdb_qlora"

trainer.save_model(adapter_path)

print("QLoRA adapter saved successfully!")
```

```
QLoRA adapter saved successfully!
```

```
import os

print(os.listdir(adapter_path))
```

```
['checkpoint-250', 'training_args.bin', 'adapter_config.json', 'README.md', 'adapter_model.safetensors', 'tokenizer_config.j
```

```
test_review = """
This movie was absolutely fantastic. The story was engaging,
the actors gave excellent performances, and I really enjoyed
every minute of it.
```

```
"""

prompt = f"""Review:
{test_review}

Sentiment:
"""

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=10
    )

response = tokenizer.decode(outputs[0], skip_special_tokens=True)

print(response)
```

```
[transformers] `use_cache=True` is incompatible with gradient checkpointing. Setting `use_cache=False`.
[transformers] Caching is incompatible with gradient checkpointing in Qwen2DecoderLayer. Setting `past_key_values=None`.
Review:

This movie was absolutely fantastic. The story was engaging,
the actors gave excellent performances, and I really enjoyed
every minute of it.


Sentiment:
Positive.
:
:
:
:
:
:
:
:
```

```
test_review = """
This movie was terrible. The story was boring and predictable,
the acting was poor, and I regretted watching it.
"""

prompt = f"""Review:
{test_review}

Sentiment:
"""

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=5
    )

response = tokenizer.decode(outputs[0], skip_special_tokens=True)

print(response)
```

```
Review:

This movie was terrible. The story was boring and predictable,
the acting was poor, and I regretted watching it.


Sentiment:
Negative av.
:
:
```

```
correct = 0
total = 20

for i in range(total):
    review = test_dataset[i]["text"]

    # Get the original sentiment
    actual = "Positive" if test_dataset[i]["label"] == 1 else "Negative"
```

```
    # Extract only the review from our formatted text
    review_text = review.split("Sentiment:")[0].replace("Review:", "").strip()

    prompt = f"""Review:
{review_text}

Sentiment:"""

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=3
        )

    generated = tokenizer.decode(
        outputs[0][inputs["input_ids"].shape[1]:],
        skip_special_tokens=True
    ).strip().lower()

    if generated.startswith("positive"):
        predicted = "Positive"
    elif generated.startswith("negative"):
        predicted = "Negative"
    else:
        predicted = "Unknown"

    if predicted == actual:
        correct += 1

    print(f"{i+1}. Actual: {actual} | Predicted: {predicted}")

accuracy = correct / total * 100

print("\n" + "=" * 40)
print(f"Correct: {correct}/{total}")
print(f"Accuracy: {accuracy:.2f}%")
print("=" * 40)
```

```
1. Actual: Positive | Predicted: Positive
2. Actual: Positive | Predicted: Positive
3. Actual: Negative | Predicted: Negative
4. Actual: Positive | Predicted: Positive
5. Actual: Negative | Predicted: Negative
6. Actual: Positive | Predicted: Positive
7. Actual: Positive | Predicted: Positive
8. Actual: Negative | Predicted: Negative
9. Actual: Negative | Predicted: Negative
10. Actual: Positive | Predicted: Positive
11. Actual: Positive | Predicted: Positive
12. Actual: Negative | Predicted: Negative
13. Actual: Negative | Predicted: Negative
14. Actual: Negative | Predicted: Negative
15. Actual: Positive | Predicted: Positive
16. Actual: Positive | Predicted: Positive
17. Actual: Negative | Predicted: Negative
18. Actual: Negative | Predicted: Negative
19. Actual: Positive | Predicted: Positive
20. Actual: Positive | Predicted: Positive

========================================
Correct: 20/20
Accuracy: 100.00%
========================================
```

```
correct = 0
unknown = 0
predictions = []

model.eval()

for i, example in enumerate(test_dataset):

    # Extract original review
    review_text = example["text"].split("Sentiment:")[0]
    review_text = review_text.replace("Review:", "").strip()

    actual = example["label"]

    prompt = f"""Review:
{review_text}
```

```
Sentiment:"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    ).to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=3,
            do_sample=False,
            pad_token_id=tokenizer.pad_token_id
        )

    generated = tokenizer.decode(
        outputs[0][inputs["input_ids"].shape[1]:],
        skip_special_tokens=True
    ).strip().lower()

    if "positive" in generated:
        predicted = 1
    elif "negative" in generated:
        predicted = 0
    else:
        predicted = None
        unknown += 1

    predictions.append(predicted)

    if predicted == actual:
        correct += 1

    if (i + 1) % 50 == 0:
        print(f"Processed: {i + 1}/500")

accuracy = correct / len(test_dataset) * 100

print("\n" + "=" * 50)
print("FINAL EVALUATION")
print("=" * 50)
print(f"Total Reviews : {len(test_dataset)}")
print(f"Correct       : {correct}")
print(f"Unknown       : {unknown}")
print(f"Accuracy      : {accuracy:.2f}%")
print("=" * 50)
```

```
Processed: 50/500
Processed: 100/500
Processed: 150/500
Processed: 200/500
Processed: 250/500
Processed: 300/500
Processed: 350/500
Processed: 400/500
Processed: 450/500
Processed: 500/500

==================================================
FINAL EVALUATION
==================================================
Total Reviews : 500
Correct       : 466
Unknown       : 0
Accuracy      : 93.20%
==================================================
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Remove any unknown predictions
valid_indices = [
    i for i, pred in enumerate(predictions)
    if pred is not None
]

actual_labels = [
    test_dataset[i]["label"]
    for i in valid_indices
]

predicted_labels = [
    predictions[i]
```

```
    for i in valid_indices
]

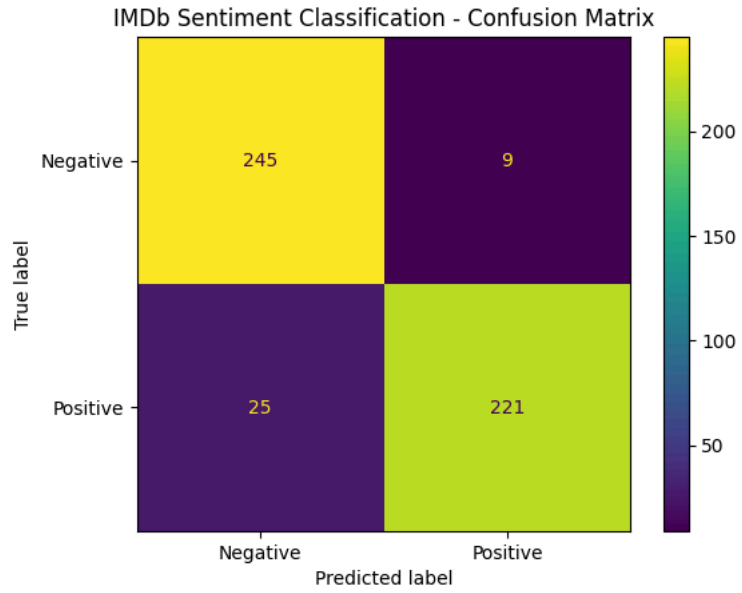
# Create confusion matrix
cm = confusion_matrix(
    actual_labels,
    predicted_labels
)

print("Confusion Matrix:")
print(cm)

# Display confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negative", "Positive"]
)

disp.plot()
plt.title("IMDb Sentiment Classification - Confusion Matrix")
plt.show()
```

```
Confusion Matrix:
[[245   9]
 [ 25 221]]
```




```
from sklearn.metrics import classification_report

print(classification_report(
    actual_labels,
    predicted_labels,
    target_names=["Negative", "Positive"]
))
```

```
              precision    recall  f1-score   support

    Negative       0.91      0.96      0.94       254
    Positive       0.96      0.90      0.93       246

    accuracy                           0.93       500
   macro avg       0.93      0.93      0.93       500
weighted avg       0.93      0.93      0.93       500
```

```
!pip install -q gradio
```

```
import gradio as gr

print("Gradio installed successfully!")
```

```
Gradio installed successfully!
```

```
!pip install -q transformers torch gradio
```

```
from transformers import pipeline
```

```
sentiment_model = pipeline(
    "sentiment-analysis",
    model="textattack/bert-base-uncased-imdb"
)

print("Model loaded successfully!")
```

Show hidden output

```
import gradio as gr
from transformers import pipeline

# ========================================
# LOAD IMDb SENTIMENT MODEL
# ========================================

sentiment_model = pipeline(
    "sentiment-analysis",
    model="textattack/bert-base-uncased-imdb"
)


# ========================================
# SENTIMENT FUNCTION
# ========================================

def analyze_sentiment(review):

    if not review or not review.strip():
        return (
            "◯ Not Analyzed",
            "Enter a movie review to get the result.",
            0,
            "Please enter a review."
        )

    result = sentiment_model(review[:1000])[0]

    label = result["label"]
    confidence = result["score"] * 100

    if label == "LABEL_1":
        sentiment = "😊 Positive"
        description = "The review expresses a positive feeling about the movie."
    else:
        sentiment = "😔 Negative"
        description = "The review expresses a negative feeling about the movie."

    return (
        sentiment,
        description,
        round(confidence, 2),
        f"{sentiment} — {confidence:.2f}% confidence"
    )


# ========================================
# CUSTOM CSS
# ========================================

css = """

/* ---------- MAIN PAGE ---------- */

.gradio-container {
    max-width: 1250px !important;
    margin: auto !important;
    background:
        radial-gradient(circle at 85% 15%, #3a1c00 0%, transparent 25%),
        radial-gradient(circle at 15% 80%, #1a0800 0%, transparent 25%),
        linear-gradient(135deg, #020303, #080b0c, #020303) !important;
    color: white !important;
    font-family: Arial, sans-serif !important;
}


/* ---------- HEADER ---------- */

.movie-header {
    text-align: center;
    padding: 25px 10px 15px 10px;
}
```

```
.imdb-logo {
    display: inline-block;
    background: #f5c518;
    color: #111;
    font-size: 28px;
    font-weight: 900;
    padding: 5px 12px;
    border-radius: 6px;
    margin-right: 12px;
}

.main-title {
    display: inline-block;
    font-size: 38px;
    font-weight: 800;
    vertical-align: middle;
}

.subtitle {
    color: #bdbdbd;
    font-size: 16px;
    margin-top: 8px;
}


/* ---------- FEATURE CARDS ---------- */

.feature-card {
    background: rgba(15, 20, 21, 0.85);
    border: 1px solid #27383d;
    border-radius: 14px;
    padding: 14px;
    text-align: center;
    color: white;
}

.feature-icon {
    font-size: 24px;
}

.feature-title {
    font-weight: bold;
    margin-top: 5px;
}

.feature-text {
    color: #aaa;
    font-size: 13px;
}


/* ---------- MOVIE SECTION ---------- */

.section-card {
    background: rgba(7, 12, 13, 0.92);
    border: 1px solid #26363b;
    border-radius: 18px;
    padding: 20px;
}


/* ---------- HEADINGS ---------- */

.section-title {
    font-size: 23px;
    font-weight: bold;
    color: white;
}

.section-title span {
    color: #f5c518;
}


/* ---------- TEXTBOX ---------- */

textarea {
    background: #080d0f !important;
    color: white !important;
    border: 1px solid #31515d !important;
    border-radius: 12px !important;
    font-size: 16px !important;
```

```
}


/* ---------- BUTTON ---------- */

.analyze-btn {
    background: linear-gradient(90deg, #f5b400, #ffd21c) !important;
    color: #111 !important;
    font-size: 18px !important;
    font-weight: bold !important;
    border: none !important;
    border-radius: 10px !important;
    height: 52px !important;
}


/* ---------- RESULT CARD ---------- */

.result-card {
    background: linear-gradient(
        145deg,
        rgba(10, 22, 20, 0.95),
        rgba(5, 12, 14, 0.95)
    );
    border: 1px solid #25443d;
    border-radius: 16px;
    padding: 22px;
    min-height: 250px;
}

.result-title {
    font-size: 23px;
    font-weight: bold;
    color: #f5c518;
}

.result-value {
    font-size: 32px;
    font-weight: bold;
    margin-top: 20px;
}

.result-description {
    color: #aaa;
    font-size: 15px;
    margin-top: 10px;
}


/* ---------- EXAMPLES ---------- */

.example-card {
    background: #0b1113;
    border: 1px solid #28424c;
    border-radius: 12px;
    padding: 15px;
}

.example-title {
    color: white;
    font-size: 18px;
    font-weight: bold;
}


/* ---------- POPCORN ---------- */

.popcorn-left {
    position: fixed;
    left: 10px;
    bottom: 25px;
    font-size: 70px;
    opacity: 0.8;
}

.popcorn-right {
    position: fixed;
    right: 10px;
    top: 100px;
    font-size: 70px;
    opacity: 0.8;
}
```

```
/* ---------- CINEMA FOOTER ---------- */

.cinema-footer {
    text-align: center;
    margin-top: 25px;
    padding: 20px;
    border-top: 1px solid #26363b;
    color: #aaa;
    letter-spacing: 4px;
}

.cinema-footer span {
    color: #f5c518;
}


/* ---------- HIDE GRADIO FOOTER ---------- */

footer {
    display: none !important;
}

"""


# ========================================
# GRADIO APP
# ========================================

with gr.Blocks(
    css=css,
    title="IMDb Movie Sentiment Analyzer"
) as demo:

    # Popcorn decorations
    gr.HTML("""
        <div class="popcorn-left">🍿</div>
        <div class="popcorn-right">🍿</div>
    """)


    # ====================================
    # HEADER
    # ====================================

    gr.HTML("""
        <div class="movie-header">

            <div>
                <span class="imdb-logo">IMDb</span>
                <span class="main-title">
                    Movie Sentiment Analyzer
                </span>
            </div>

            <div class="subtitle">
                Discover what people really feel about movies
                using AI and Natural Language Processing.
            </div>

        </div>
    """)


    # ====================================
    # FEATURES
    # ====================================

    with gr.Row():

        gr.HTML("""
        <div class="feature-card">

            <div class="feature-icon">🧠</div>

            <div class="feature-title">
                AI Powered
            </div>

            <div class="feature-text">
                BERT Model
```

```
            </div>

        </div>
        """)

        gr.HTML("""
        <div class="feature-card">

            <div class="feature-icon">😊</div>

            <div class="feature-title">
                Sentiment Detection
            </div>

            <div class="feature-text">
                Positive / Negative
            </div>

        </div>
        """)

        gr.HTML("""
        <div class="feature-card">

            <div class="feature-icon">⚡</div>

            <div class="feature-title">
                Instant Results
            </div>

            <div class="feature-text">
                Fast Prediction
            </div>

        </div>
        """)


    gr.Markdown("")


    # ======================================
    # MAIN ANALYZER
    # ======================================

    with gr.Row():

        # ---------------------------------
        # LEFT SIDE
        # ---------------------------------

        with gr.Column(scale=1):

            gr.HTML("""
                <div class="section-card">

                    <div class="section-title">
                        🎬 <span>Enter Movie Review</span>
                    </div>

                    <p style="color:#aaa;">
                        Type or paste a movie review below...
                    </p>

                </div>
            """)

            review_input = gr.Textbox(
                placeholder=(
                    "e.g. This movie was absolutely amazing! "
                    "The acting, story and visuals were top notch..."
                ),
                lines=9,
                max_lines=12,
                show_label=False
            )

            analyze_button = gr.Button(
                "🔍  Analyze Sentiment",
                elem_classes="analyze-btn"
            )
```

```
        # ---------------------------------
        # RIGHT SIDE
        # ---------------------------------

        with gr.Column(scale=1):

            gr.HTML("""
                <div class="result-card">

                    <div class="result-title">
                        ✨ Sentiment Result
                    </div>

                </div>
            """)

            sentiment_output = gr.Textbox(
                value="◯ Not Analyzed",
                label="Sentiment",
                interactive=False
            )

            description_output = gr.Textbox(
                value="Enter a movie review to get the result.",
                label="Analysis",
                interactive=False
            )

            confidence_output = gr.Number(
                value=0,
                label="Confidence (%)",
                interactive=False
            )


    # =====================================
    # EXAMPLE REVIEWS
    # =====================================

    gr.Markdown("")

    gr.HTML("""
        <div class="example-card">

            <div class="example-title">
                💬 Try these example reviews
            </div>

        </div>
    """)

    gr.Examples(
        examples=[
            [
                "Great movie! Loved the performances and story."
            ],
            [
                "Terrible movie. Waste of time and money."
            ],
            [
                "It was okay, nothing special."
            ],
            [
                "An amazing cinematic experience!"
            ]
        ],
        inputs=review_input
    )


    # =====================================
    # FOOTER
    # =====================================

    gr.HTML("""
        <div class="cinema-footer">

            🎞️ <span>MOVIES</span>
              •  
            <span>REVIEWS</span>
              •  
            <span>SENTIMENTS</span>
              🏠
```

```
        </div>
    """)


    # ======================================
    # BUTTON ACTION
    # ======================================

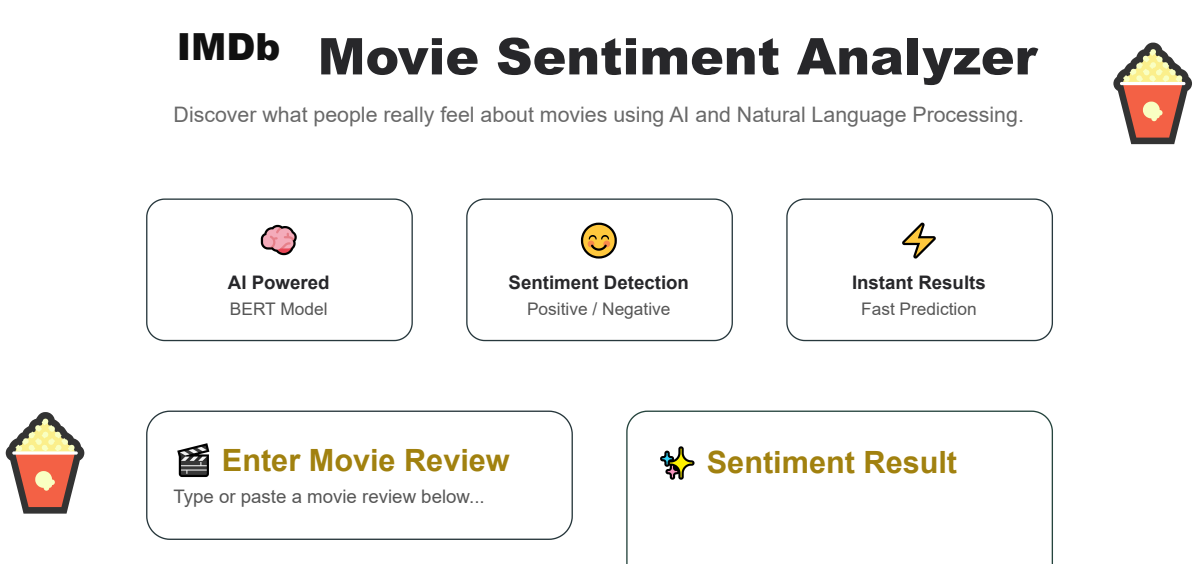
    analyze_button.click(
        fn=analyze_sentiment,
        inputs=review_input,
        outputs=[
            sentiment_output,
            description_output,
            confidence_output
        ]
    )


# ========================================
# LAUNCH
# ========================================

demo.launch(share=True)
```

```
Loading weights: 100%                                    201/201 [00:00<00:00, 4322.56it/s]
/tmp/ipykernel_1833/738981232.py:270: UserWarning: The parameters have been moved from the Blocks constructor to the launch(
  with gr.Blocks(
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://f2f6dfb02ec63b6ae4.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run
```



Next steps: Deploy to Cloud Run

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.